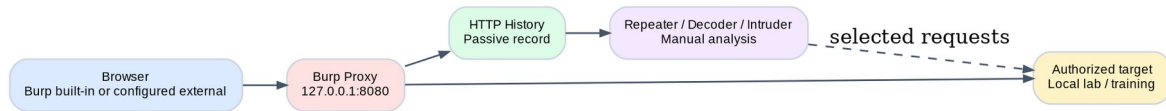


## RED TEAM LEARNING PATH

## MODULE 10

# Burp Suite Fundamentals

Proxy, HTTP History, Repeater, Scope, TLS, Decoder & Controlled Automation



*Burp sits between your browser and an authorized target; interesting traffic is promoted into analysis tools.*

**Authorized-lab only**

Semua hands-on di modul ini ditujukan untuk localhost, container milik sendiri, Web Security Academy, atau target yang secara eksplisit mengizinkan pengujian. Jangan memasukkan host pihak lain ke scope Burp tanpa izin.

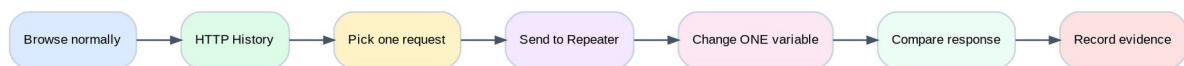
Omarchy / Arch Linux Edition | v1.0 | 04 Oct 2026

| Field        | Value                                                                                                           |
|--------------|-----------------------------------------------------------------------------------------------------------------|
| Prerequisite | Module 00–09 — terutama 03-http-web-fundamentals, 06-pentest-methodology-scope-roe, 09-vulnerability-assessment |
| Core tools   | Burp Suite Community Edition, Burp built-in browser, curl, Python 3, Docker (opsional)                          |
| Local target | http://127.0.0.1:8090/ (lab utama),<br>http://127.0.0.1:3000/ (Juice Shop, opsional)                            |
| Outcome      | Terbiasa membaca & mengontrol raw HTTP request/response secara manual, dengan evidence yang reproducible        |

## 1. Learning Objectives

- [ ] Menjelaskan posisi Burp Proxy di antara browser dan aplikasi.
- [ ] Menggunakan Burp built-in browser, HTTP History, Intercept, Repeater, Decoder, dan konsep dasar Intruder.
- [ ] Menetapkan **target scope** sejak awal untuk membatasi noise/out-of-scope traffic.
- [ ] Mengamati cookie/session tanpa menganggap cookie sebagai sekadar string.
- [ ] Menjelaskan cara kerja HTTPS interception dan risiko memasang Burp CA ke browser eksternal.
- [ ] Membandingkan response setelah mengubah **satu variabel** di Repeater.
- [ ] Mencatat evidence dan observasi secara reproducible.

## 2. Mental Model: Burp Bukan "Scanner Ajaib"



*Workflow manual yang akan kita pakai berulang kali sampai module web exploitation.*

Burp Proxy adalah **perantara**: browser mengirim request ke listener Burp, Burp meneruskan ke server, response kembali lewat Burp. Nilai utamanya adalah *visibility* dan *controlled modification* — bukan tombol "find vulnerability".

```

Browser → [Burp Proxy :8080] → Target
      ↓
  HTTP History / Intercept / Repeater / Decoder / Intruder
  
```

### Red Team perspective.

Tool bukan tujuan. Pertanyaan yang lebih penting: request mana yang menarik, parameter apa yang dikontrol user, state apa yang dibawa cookie/token, dan apa yang berubah ketika satu input dimodifikasi?

## 3. Install Burp Suite di Omarchy

Gunakan **Burp Suite Community Edition** dari installer resmi PortSwigger — bukan package AUR acak, karena tool ini memproses traffic sensitif selama testing.

```

cd ~/Downloads
sha256sum burpsuite_community_linux_*.sh
chmod +x burpsuite_community_linux_*.sh
./burpsuite_community_linux_*.sh
  
```

- Download dari halaman resmi PortSwigger → *Burp Suite Community Edition*.
- Pilih installer Linux sesuai arsitektur device.
- Hitung hash lokal (sha256sum) untuk provenance/evidence sebelum menjalankan installer.
- Jalankan sebagai user biasa — sudo hanya kalau installer memang minta privilege untuk lokasi system-wide.

- Pada first run, pilih *temporary project* / default configuration untuk latihan awal.

#### Hash note.

Hash yang lo hitung sendiri berguna untuk membuktikan file yang sama dipakai kembali (provenance lokal). Hash itu **bukan** verifikasi vendor kalau tidak dibandingkan dengan nilai resmi yang dipublikasikan PortSwigger.

## 4. Built-in Browser vs External Browser

| Mode             | Kelebihan                              | Risiko/trade-off                              |
|------------------|----------------------------------------|-----------------------------------------------|
| Built-in browser | Langsung bekerja; isolated workflow    | Profile terpisah dari browser harian          |
| External browser | Bisa menyerupai workflow browser utama | Perlu proxy config; HTTPS butuh trust Burp CA |

#### Security note.

Jangan pasang Burp CA ke trust store system-wide hanya demi latihan. Kalau memang perlu external browser, pakai profile browser khusus testing. Trusted root CA punya kemampuan MITM terhadap semua koneksi yang dipercaya profile itu (lihat §11).

## 5. Proxy Listener

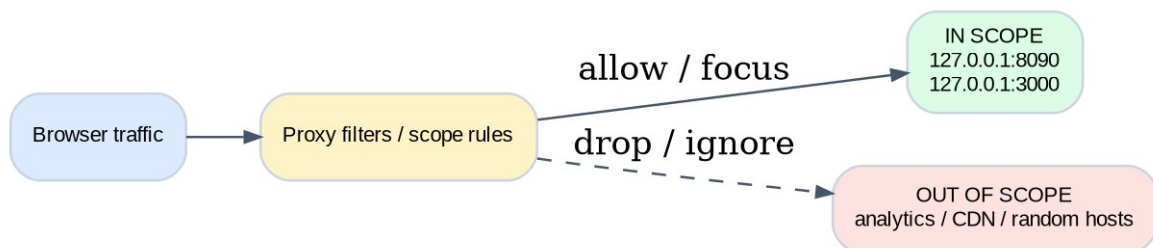
Default Burp membuat listener di **loopback port 8080**. Loopback penting di sini — proxy tidak perlu terekspos ke LAN hanya untuk testing browser lokal.

```
ss -ltnp | grep ':8080' || true
```

#### Common mistake.

Jangan bind Burp listener ke 0.0.0.0 hanya karena "biar gampang". Itu memperluas attack surface workstation lo dan bisa membuat proxy diakses device lain di jaringan yang sama.

## 6. Target Scope: Safety Boundary Pertama



*Contoh scope Module 10: hanya service loopback yang kita miliki.*

Set scope **sebelum** mulai testing. Burp bisa memfilter Site map dan HTTP History berdasarkan scope, mengurangi kemungkinan tools (termasuk Intruder/Repeater) mengirim request ke host yang tidak dimaksud.

| Scope item                | Status | Alasan                               |
|---------------------------|--------|--------------------------------------|
| http://127.0.0.1:8090/    | IN     | Custom Module 10 lab                 |
| http://127.0.0.1:3000/    | IN     | Optional Juice Shop container        |
| analytics/CDN/domain lain | OUT    | Tidak dibutuhkan untuk latihan lokal |

Guardrail helper untuk URL training (menolak target non-loopback & port di luar daftar 8090/3000/8080):

```
python labs/scope-guard.py http://127.0.0.1:8090/
python labs/scope-guard.py http://127.0.0.1:3000/
```

### Important.

"Ada di HTTP History" **tidak sama** dengan "boleh diuji". Browser modern menghubungi banyak third-party endpoint tanpa sepengetahuan eksplisit lo. Authorization tetap berasal dari scope/RoE engagement — bukan dari apa yang kebetulan lewat Burp.

## 7. Proxy > HTTP History

HTTP History menyimpan traffic yang lewat Proxy **bahkan saat Intercept dimatikan**. Ini cocok untuk mode kerja normal: browse aplikasi tanpa gangguan, lalu pilih request menarik setelahnya.

- Filter ke in-scope traffic.
- Perhatikan method, URL, status code, MIME type, cookies, response length, dan TLS.
- Klik entry untuk membaca raw request dan response.
- Pakai notes kalau perlu, supaya request penting tidak hilang di antara noise.

### Blue Team perspective.

Proxy history juga mengingatkan bahwa setiap request meninggalkan pola: method, path, timing, cookies, sequence. Saat nanti masuk Module 24 — OPSEC & Detection-Aware Operations, pikirkan telemetry yang dihasilkan oleh testing lo sendiri.

## 8. Intercept: Tahan, Ubah, Forward, Drop

Intercept cocok kalau lo mau memodifikasi request **sebelum** server melihatnya. Tapi browser menghasilkan banyak request sekaligus, jadi Intercept sebaiknya dipakai **secara sengaja**, bukan selalu ON.

| Action           | Makna                                          |
|------------------|------------------------------------------------|
| Forward          | Teruskan request yang sedang ditahan           |
| Drop             | Jangan kirim request ke server                 |
| Edit raw message | Ubah method/header/body sebelum forward        |
| Send to Repeater | Salin request untuk eksperimen manual berulang |

### Common mistake.

Kalau halaman terasa "hang", cek dulu apakah Intercept masih ON dan ada request yang sedang ditahan — salah satu sumber kebingungan paling umum untuk pemula.

## 9. Repeater: Laboratory Bench untuk HTTP

Repeater memungkinkan satu request dikirim berulang dengan perubahan terkontrol — tool utama untuk membangun hipotesis secara manual.

### Rule of one.

Ubah **SATU** variabel per eksperimen kalau memungkinkan. Kalau lo mengganti method, cookie, parameter, dan header sekaligus, lo kehilangan kemampuan menjelaskan penyebab perubahan response.

| Baseline             | Change         | Bandingkan                |
|----------------------|----------------|---------------------------|
| GET /echo?name=alice | name=bob       | status, body, length      |
| Cookie absent        | Cookie present | state/response difference |

|                  |                  |                           |
|------------------|------------------|---------------------------|
| POST JSON valid  | JSON malformed   | validation behavior       |
| Redirect request | follow/no-follow | Location + final response |

Contoh baseline request lokal:

```
GET /echo?name=alice&role=user HTTP/1.1
Host: 127.0.0.1:8090
User-Agent: Burp-Training
Connection: close
```

## 10. Decoder dan Encoding

Burp Decoder membantu encode/decode URL encoding, HTML encoding, Base64, hex, dan transformasi lainnya. **Encoding bukan encryption** — tujuan utamanya representasi data, bukan secrecy.

```
Original:    role=user admin
URL encoded: role%3Duser%20admin
Base64:     cm9sZT11c2VyIGFkbWlu
```

### Security note.

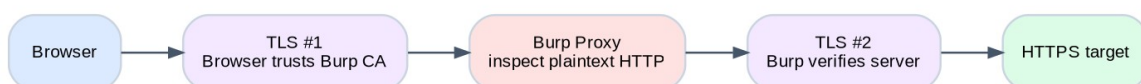
Jangan menganggap Base64, URL encoding, atau hex sebagai perlindungan secret. Kalau token sensitif cuma "disembunyikan" dengan encoding, siapa pun yang mendapatkannya bisa decode dalam sekali klik.

## 11. Cookies, Session, dan State

Burp sangat membantu melihat state yang dibawa browser. Perhatikan Set -Cookie pada response dan Cookie pada request berikutnya. **Jangan edit token production atau akun real** di luar scope.

| Attribute     | Pertanyaan pentest dasar                      |
|---------------|-----------------------------------------------|
| Secure        | Apakah cookie hanya dikirim lewat HTTPS?      |
| HttpOnly      | Apakah JavaScript seharusnya bisa membacanya? |
| SameSite      | Bagaimana cross-site delivery dibatasi?       |
| Path / Domain | Ke request mana cookie berlaku?               |
| Lifetime      | Session cookie atau persistent cookie?        |

## 12. HTTPS Interception dan Burp CA



*HTTPS interception memutus satu koneksi TLS menjadi dua koneksi TLS terpisah.*

Untuk membaca HTTPS, Burp membuat **dua koneksi TLS terpisah**: browser ↔ Burp dan Burp ↔ server. Browser harus mempercayai CA milik Burp supaya tidak ada warning saat pakai external browser.

```
Browser —TLS #1—> Burp —TLS #2—> Server
      (Burp cert)      (real server cert)
```

### Prefer built-in browser.

PortSwigger menyediakan browser bawaan yang sudah dikonfigurasi — pakai itu dulu untuk module ini. External browser + instalasi CA baru diperlukan kalau ada alasan jelas.

**CA private key matters.**

Kalau private key CA yang dipercaya browser itu bocor, siapa pun yang memegangnya bisa membuat certificate yang dipercaya profile tersebut. Jaga workstation lo, pakai profile testing terpisah, dan jangan menyalin CA/private key sembarangan.

## 13. Intruder Concepts — Controlled Automation

Intruder mengulang request dan memasukkan payload ke posisi tertentu. Fitur ini powerful — Module 10 **hanya** memakai payload kecil terhadap loopback lab. Brute-force, credential attack, dan aggressive fuzzing **bukan** objective module ini (lihat Module 15 untuk itu, dalam konteks yang eksplisit authorized).

| Konsep           | Makna                                      |
|------------------|--------------------------------------------|
| Payload position | Bagian request yang akan diganti           |
| Payload set      | Daftar nilai yang dicoba                   |
| Attack type      | Cara payload dipetakan ke posisi           |
| Resource/rate    | Seberapa cepat request dikirim             |
| Result columns   | Status, length, timing, grep/extract field |

**Safety budget.**

Untuk lab pertama cukup 3–5 payload ke endpoint localhost. Tujuannya memahami mekanisme dan membandingkan response — bukan menghasilkan traffic sebanyak mungkin.

## 14. Evidence Discipline

Burp membantu menemukan behavior, tapi report tetap butuh evidence yang bisa direproduksi.

| Field       | Contoh                                                   |
|-------------|----------------------------------------------------------|
| Request     | GET /echo?name=alice                                     |
| Tool        | Proxy History → Repeater                                 |
| Change      | name=alice → name=bob                                    |
| Observation | response JSON mengikuti parameter                        |
| Evidence    | exported message / screenshot / hash                     |
| Conclusion  | behavior confirmed; <i>belum</i> ada vulnerability claim |

```
python labs/observation-log.py \
  module10-burp/04-notes/observations.csv \
  --tool Repeater \
  --request 'GET /echo?name=bob' \
  --observation 'Response JSON reflected name=bob'
```

observation-log.py juga menerima --evidence <file> opsional untuk menyertakan SHA-256 file bukti di kolom evidence\_sha256.

## 15. Lab Setup

Lab utama adalah service Python loopback (burp-lab-server.py) dengan endpoint /echo, /cookie, /redirect, /login, /reflect. Juice Shop bersifat opsional dan tetap bind hanya ke 127.0.0.1 supaya tidak terekspos ke LAN.

```
cd modules/10-burp-suite-fundamentals/labs
./workspace-init.sh module10-burp
python burp-lab-server.py --port 8090
```

Server menolak bind ke host selain 127.0.0.1:::1/localhost dan port di bawah 1024 — guardrail bawaan di burp-lab-server.py sendiri, bukan cuma dokumentasi.

## 16. Hands-on Lab 10.1 — Workspace dan Scope

```
./workspace-init.sh module10-burp
cat module10-burp/00-scope/scope.txt
python scope-guard.py http://127.0.0.1:8090/
```

[ ] Workspace permission private (umask 077).

[ ] Scope hanya loopback training URLs.

[ ] Semua host lain dianggap OUT sampai ada authorization terpisah.

## 17. Hands-on Lab 10.2 — Start Local HTTP Lab

```
python burp-lab-server.py --port 8090
```

Di terminal lain:

```
curl -i http://127.0.0.1:8090/
curl -i 'http://127.0.0.1:8090/echo?name=alice&role=user'
```

### Expected.

Server bind hanya ke 127.0.0.1 dan menyediakan endpoint /echo, /cookie, /redirect, /login, /reflect untuk observasi protocol.

## 18. Hands-on Lab 10.3 — Proxy + HTTP History

- Buka Burp → Proxy → Intercept → *Open Browser*.
- Pastikan Intercept **OFF** agar browsing lancar.
- Buka `http://127.0.0.1:8090/`.
- Kunjungi `/echo?name=alice&role=user`, `/cookie`, dan `/redirect`.
- Buka Proxy → HTTP history dan filter traffic ke 127.0.0.1:8090.
- Catat method, status, length, MIME type, cookies, dan redirect.

## 19. Hands-on Lab 10.4 — Set Target Scope

- Dari HTTP History atau Site map, *add* `http://127.0.0.1:8090/` ke scope.
- Filter tampilan ke in-scope items.
- Pastikan target lain tidak ikut diproses dalam workflow module.

## 20. Hands-on Lab 10.5 — Intercept Satu Request

- Aktifkan Intercept **ON**.
- Di Burp browser buka `/echo?name=alice&role=user`.
- Saat request tertahan, ubah `name=alice` menjadi `name=bob`.
- Forward request.
- Matikan Intercept kembali.
- Bandingkan response dan catat perubahannya.

**Goal.**

Buktikan hubungan *request* → *modification* → *response*. Jangan mengejar "bug" — fokus ke kemampuan membaca dan mengontrol message.

## 21. Hands-on Lab 10.6 — Repeater Baseline

- Dari HTTP History pilih request /echo lalu *Send to Repeater*.
- Kirim baseline tanpa perubahan.
- Ubah hanya parameter name, kirim lagi.
- Ubah hanya role, kirim lagi.
- Bandingkan status, response body, dan length.

```
python observation-log.py \
  module10-burp/04-notes/observations.csv \
  --tool Repeater \
  --request 'GET /echo?name=bob&role=user' \
  --observation 'Only name changed; JSON query value changed accordingly'
```

## 22. Hands-on Lab 10.7 — Cookie Observation

- Kunjungi /cookie.
- Di response, cari Set-Cookie (akan terlihat module10\_session=training-only; Path=/; HttpOnly; SameSite=Lax).
- Kunjungi /echo lagi dan lihat header Cookie yang dikirim browser.
- Identifikasi HttpOnly, SameSite, Path, dan apakah Secure ada.
- Jangan menyimpulkan vulnerability hanya dari satu attribute tanpa context transport/application.

## 23. Hands-on Lab 10.8 — Decoder

- Pilih teks role=user admin dan *Send to Decoder*.
- URL-encode value tersebut.
- Base64-encode value aslinya.
- Decode kembali untuk membuktikan reversibility.
- Catat perbedaan encoding vs encryption di notes lo.

## 24. Hands-on Lab 10.9 — Small Intruder Demo

Gunakan hanya endpoint loopback. Dari request /echo?name=alice, kirim ke Intruder dan tandai hanya value alice sebagai payload position.

```
Payload set:
alice
bob
charlie
```

- Jalankan attack kecil dengan tiga payload.
- Bandingkan status dan response length.
- Jangan tambahkan payload besar atau high-rate configuration.
- Tutup attack setelah memahami mekanismenya.

## 25. Hands-on Lab 10.10 — Optional Juice Shop Proxy Workflow

```
./juice-shop-lab.sh create
./juice-shop-lab.sh status
```

- Add `http://127.0.0.1:3000/` ke scope.
- Browse halaman dengan Intercept OFF.
- Gunakan HTTP History untuk mengenali XHR/fetch/API request.
- Pilih **satu** harmless GET request dan kirim ke Repeater.
- Jangan mengerjakan challenge exploitation dulu — objective-nya hanya tool workflow.

```
./juice-shop-lab.sh cleanup
```

## 26. Practical Challenge — "Map One Interaction"

Tanpa walkthrough, pilih satu interaction dari local Module 10 lab dan dokumentasikan alur lengkapnya.

- ☐ Scope URL ditulis jelas.
- ☐ Baseline request disimpan/dicatat.
- ☐ HTTP History entry diidentifikasi.
- ☐ Request dikirim ke Repeater.
- ☐ Hanya satu variabel diubah.
- ☐ Response dibandingkan secara eksplisit.
- ☐ Cookie/state dicatat bila relevan.
- ☐ Tidak ada klaim vulnerability tanpa bukti impact.
- ☐ Evidence/notes punya timestamp dan hash (kalau ada file evidence).

### Definition of done.

Orang lain harus bisa membaca notes lo dan mengulangi eksperimen yang sama terhadap local lab untuk mendapatkan behavior yang setara.

## 27. Knowledge Check

- Apa perbedaan HTTP History dengan Intercept?
- Kenapa Intercept tidak perlu selalu ON?
- Apa manfaat menetapkan Target Scope sebelum testing?
- Kenapa traffic yang muncul di Burp tidak otomatis berarti authorized?
- Kapan Repeater lebih berguna daripada browser biasa?
- Kenapa mengubah satu variabel per eksperimen membantu reasoning?
- Apa perbedaan encoding dengan encryption?
- Bagaimana Set-Cookie berbeda dari Cookie?
- Kenapa Burp built-in browser cocok untuk pemula?
- Kenapa external browser butuh Burp CA untuk HTTPS?
- Apa risiko dari trusted root CA yang bocor?
- Apa yang dimaksud *payload position* di Intruder?
- Kenapa Intruder dibatasi ke payload kecil di module ini?
- Bagaimana cara membuktikan sebuah evidence file tidak berubah?
- Kapan lo boleh mulai menguji target non-local?

## 28. Exit Criteria

- [ ] Bisa menjalankan Burp Community Edition di Omarchy.
- [ ] Memahami listener loopback dan built-in browser.
- [ ] Bisa membuat scope dan memfilter history.
- [ ] Bisa intercept satu request dan mengembalikannya ke flow normal.
- [ ] Bisa menggunakan Repeater untuk baseline/change/compare.
- [ ] Bisa menggunakan Decoder untuk transformasi encoding dasar.
- [ ] Bisa menjelaskan cookie/session observation.
- [ ] Memahami konsep TLS interception dan Burp CA.
- [ ] Memahami konsep Intruder tanpa melakukan brute-force.
- [ ] Menyimpan notes/evidence yang reproducible.

## 29. Deliverables

- docx/ — editable source
- pdf/ — final learning module
- labs/workspace-init.sh — scaffold workspace private (00-scope ... 05-summary)
- labs/scope-guard.py — guardrail URL training (loopback + port whitelist)
- labs/burp-lab-server.py — local HTTP training server (/echo, /cookie, /redirect, /login, /reflect)
- labs/juice-shop-lab.sh — optional Juice Shop lifecycle wrapper
- labs/observation-log.py — structured observation log (CSV + SHA-256 evidence opsional)

## 30. Cheat Sheet

| Tujuan                 | Lokasi/aksi                                               |
|------------------------|-----------------------------------------------------------|
| Buka browser Burp      | Proxy → Intercept → <i>Open Browser</i>                   |
| Lihat semua traffic    | Proxy → HTTP history                                      |
| Tetapkan scope         | Target → Scope / klik kanan request → <i>Add to scope</i> |
| Tahan request          | Proxy → Intercept → Intercept ON                          |
| Eksperimen manual      | <i>Send to Repeater</i>                                   |
| Transform encoding     | <i>Send to Decoder</i>                                    |
| Controlled automation  | <i>Send to Intruder</i>                                   |
| Default listener       | 127.0.0.1:8080                                            |
| Local Module 10 target | 127.0.0.1:8090                                            |
| Optional Juice Shop    | 127.0.0.1:3000                                            |

```

./labs/workspace-init.sh module10-burp
python labs/burp-lab-server.py --port 8090
python labs/scope-guard.py http://127.0.0.1:8090/
./labs/juice-shop-lab.sh create # optional
./labs/juice-shop-lab.sh cleanup

```

## 31. References

- PortSwigger — Download and install Burp Suite — <https://portswigger.net/burp/documentation/desktop/getting-started/download-and-install>
- PortSwigger — Burp Proxy — <https://portswigger.net/burp/documentation/desktop/tools/proxy>
- PortSwigger — HTTP History — <https://portswigger.net/burp/documentation/desktop/tools/proxy/http-history>
- PortSwigger — Target Scope — <https://portswigger.net/burp/documentation/desktop/tools/target/scope>
- PortSwigger — Burp Repeater — <https://portswigger.net/burp/documentation/desktop/tools/repeater>
- PortSwigger — Burp Decoder — <https://portswigger.net/burp/documentation/desktop/tools/decoder>
- PortSwigger — Burp Intruder — <https://portswigger.net/burp/documentation/desktop/tools/intruder>
- PortSwigger — Installing Burp CA certificate — <https://portswigger.net/burp/documentation/desktop/external-browser-config/certificate>
- OWASP Developer Guide — Juice Shop — <https://devguide.owasp.org/en/07-training-education/01-vulnerable-apps/01-juice-shop/>
- OWASP Juice Shop project — <https://owasp.org/projects/juice-shop>

### Next module:

11-web-vulnerabilities-i — SQL Injection, XSS, Command Injection, Path Traversal, File Upload, Authentication flaws, dan Access Control/IDOR, dengan PortSwigger labs dan local authorized targets.